

Compile Expression	Instructions
$\text{compile}(\pi_{\alpha_{n,m}}(\epsilon), D)$ “Project”	$\text{let } ([s_1, \dots, s_n, s_t], c) = \text{compile}(\epsilon, D) \text{ in}$ $(c \cdot \{$ $[d_1, \dots, d_m, d_t] \leftarrow \text{alloc}(\text{size}(s_1))$ $[d_1, \dots, d_m] \leftarrow \text{eval}(\alpha_{n,m})([s_1, \dots, s_n])$ $d_t \leftarrow \text{copy}(s_t)\}, [d_n, d_t])$
$\text{compile}(\rho(x_1, \dots, x_n), D)$ “Relation”	$(\{\text{alloc}([s_1, \dots, s_n, s_t], \text{size}(D(\rho)))$ $[\overline{s_n}, s_t] \leftarrow \text{load}(D(\rho))(\cdot)\}, [\overline{s_n}, s_t])$
$\text{compile}(p \leftarrow \epsilon, D)$ “Update”	$\text{let } ([s_1, \dots, s_n, s_t], c) = \text{compile}(\epsilon) \text{ in}$ $(c \cdot \{$ $\text{store}(p, [s_1, \dots, s_n, s_t]), \emptyset)$
$\text{compile}(p_1 \leftarrow \epsilon_1, \dots, p_n \leftarrow \epsilon_n)$ “Stratum”	$\text{let } (_, c_1) = \text{compile}(p_1 \leftarrow \epsilon_1) \text{ in}$ $\dots$ $\text{let } (_, c_n) = \text{compile}(p_n \leftarrow \epsilon_n) \text{ in}$ $(c_1 \cdot \dots \cdot c_n \cdot \{$ $\text{alloc}(\overline{s_n}, \text{size}(F_T^{\text{stable}})(\rho))$ $\overline{s_n} \leftarrow \text{load}(F_T^{\text{stable}}(\rho))(\cdot)$ $\text{alloc}(\overline{r_n}, \text{size}(F_T^{\text{recent}})(\rho))$ $\overline{r_n} \leftarrow \text{load}(F_T^{\text{recent}}(\rho))(\cdot)$ $\text{alloc}(\overline{d_n}, \text{size}(F_T^\Delta(\rho)))$ $\overline{d_n} \leftarrow \text{load}(F_T^\Delta(\rho))(\cdot)$ $\text{alloc}(\overline{s_n^{\text{new}}}, \text{size}(\overline{s_n} + \overline{r_n}))$ $\overline{s_n^{\text{new}}} \leftarrow \text{merge}(\overline{s_n}, \overline{r_n})$ $\text{alloc}([\overline{d_n^{\text{sorted}}}, \overline{d_n^{\text{unique}}}], \text{size}(\overline{d_n}))$ $\overline{d_n^{\text{sorted}}} \leftarrow \text{sort}(\overline{d_n})$ $\overline{d_n^{\text{unique}}} \leftarrow \text{unique}(\overline{d_n^{\text{sorted}}})$ $\text{store}(F_T^{\text{stable}}(\rho))(\overline{s_n^{\text{new}}})$ $\text{store}(F_T^{\text{recent}}(\rho))(\overline{d_n^{\text{unique}}})$ $\text{store}(F_T^\Delta(\rho))(\emptyset)\}$ for ρ in $\text{unique}(\rho_1, \dots, \rho_n)$ $, \emptyset)$
$\text{compile}(\epsilon_1 \bowtie_w \epsilon_2, D)$ “Join”	$\text{let } (\overline{w_n}, c_1) = \text{join}_{\text{impl}}(\epsilon_1 \bowtie_w \epsilon_2, F_T^{\text{stable}}, F_T^{\text{recent}}) \text{ in}$ $\text{let } (\overline{x_n}, c_2) = \text{join}_{\text{impl}}(\epsilon_1 \bowtie_w \epsilon_2, F_T^{\text{recent}}, F_T^{\text{stable}}) \text{ in}$ $\text{let } (\overline{y_n}, c_3) = \text{join}_{\text{impl}}(\epsilon_1 \bowtie_w \epsilon_2, F_T^{\text{recent}}, F_T^{\text{recent}}) \text{ in}$ $(\{\text{alloc}(\overline{z_n}, \text{size}(\overline{w_1}) + \text{size}(\overline{x_1}) + \text{size}(\overline{y_1}))$ $\overline{z_n} \leftarrow \text{append}(\overline{w_n}, \overline{x_n}, \overline{y_n})\}, \overline{z_n})$
$\text{join}_{\text{impl}}(\epsilon_1 \bowtie_w \epsilon_2, D_1, D_2)$	$\text{let } ([a_1, \dots, a_n, a_t], c_1) = \text{compile}(\epsilon_1, D_1) \text{ in}$ $\text{let } ([b_1, \dots, b_m, b_t], c_2) = \text{compile}(\epsilon_2, D_2) \text{ in}$ $(c_1 \cdot c_2 \cdot \{$ $\text{alloc}(h, \text{size}(a_1) * O)$ $\text{static } h \leftarrow \text{build}([a_1, \dots, a_w])$ $\text{alloc}([c, o], \text{size}(b_1))$ $c \leftarrow \text{count}([b_1, \dots, b_w], h, [a_1, \dots, a_w])$ $o \leftarrow \text{scan}(c)$ $\text{alloc}([i_l, i_r, d_1, \dots, d_{n+m-w}, d_t], \text{last}(o))$ $[i_l, i_r] \leftarrow \text{join}(w)(\overline{b_m}, \overline{a_n}, h, c, o)$ $[d_1, \dots, d_n] \leftarrow \text{gather}(i_l, \overline{a_n})$ $[d_{n+1}, \dots, d_{n+m-w}] \leftarrow \text{gather}(i_r, \overline{b_m})$ $d_t \leftarrow \text{gather}(\otimes)([i_l, i_r], [a_t, b_t]), [\overline{d_n}, d_t])$

Figure 14. A subset of the function $\text{compile} :: \text{RAM} \rightarrow [\text{instr}] \times [\text{reg}]$ which translates a RAM program to APM via a per-RAM operator translation rules. We assume register names are created from fresh symbols and never conflict. We use $\cdot$ to denote sequential composition of instructions and $\leftarrow$ to denote assignment. Translation proceeds in the context of a database that is partitioned into three components: F_T^{stable} , F_T^{recent} , and F_T^Δ . This enables semi-naive evaluation, as discussed in Section 3.4.